Activity Planner — Complete Project Guide

> A full walkthrough of every file, every class, every process, and how they connect.

Table of Contents

1. [Big Picture Architecture](#1-big-picture-architecture)
2. [How a Request Flows End-to-End](#2-how-a-request-flows-end-to-end)
3. [How Kafka Events Flow End-to-End](#3-how-kafka-events-flow-end-to-end)
4. [Database Schema](#4-database-schema)
5. [Infrastructure Files](#5-infrastructure-files)
6. [auth-service (Port 8081)](#6-auth-service-port-8081)
7. [user-service (Port 8082)](#7-user-service-port-8082)
8. [activity-service (Port 8083)](#8-activity-service-port-8083)
9. [planner-service (Port 8084)](#9-planner-service-port-8084)
10. [notification-service (Port 8085)](#10-notification-service-port-8085)
11. [api-gateway (Port 8080)](#11-api-gateway-port-8080)
12. [Frontend (React SPA)](#12-frontend-react-spa)
13. [Kafka Infrastructure Classes](#13-kafka-infrastructure-classes)
14. [Kubernetes Manifests](#14-kubernetes-manifests)
15. [Documentation Files](#15-documentation-files)
16. [Test Files](#16-test-files)
17. [Process Reference: What Happens When…](#17-process-reference-what-happens-when)
18. [Environment Variables Reference](#18-environment-variables-reference)
19. [How to Build and Run](#19-how-to-build-and-run)

---

1. Big Picture Architecture

```
Browser (React SPA)
        │  HTTP  (port 8080)
        ▼
  ┌──────────────┐
  │ api-gateway │   ← single entry point; proxies all requests
  └──────┬───────┘
         │ HTTP  (internal Docker network)
    ┌────┴──────────────────────────────────────┐
    │    │              │           │             │
    ▼    ▼              ▼           ▼             ▼
 auth  user        activity      planner    notification
 8081  8082          8083          8084         8085
    │    │              │           │             │
```

```
    └────┴──────────────┴──────────┴──────────────┘
                                  │
                        MySQL (port 3306)
                        single database: activity_planner
                        tables: users, user_profiles,
                                activities, plans,
                                notifications,
                                outbox_events,
                                consumed_events

              Kafka (Redpanda, port 9092)
              topics: users.events.v1
                      activities.events.v1
                      plans.events.v1

    PRODUCERS                     CONSUMERS
    auth-service  ──────────────► user-service
    activity-svc                   (UserRegisteredConsumer)
    planner-svc   ──────────────► notification-service
    (via outbox)                   (PlanCreatedConsumer)
```

Technology stack:

Layer	Technology
Backend services	Java 11, Eclipse Vert.x 4.0 (non-blocking, event-loop)
Message broker	Apache Kafka (Redpanda in dev/K8s)
Database	MySQL 8.0
Frontend	React 17, React Router 5, Axios
Containerisation	Docker, Docker Compose
Orchestration	Kubernetes (manifests in `/k8s/`)
Build tool	Apache Maven (fat JAR via maven-shade-plugin)

---

2. How a Request Flows End-to-End

```
1. Browser calls:  POST http://localhost:8080/api/plans
                                  │
2. api-gateway receives it        │
   matches /api/plans/*  ─────────┘
   forwards to planner-service:8084/api/plans
                                  │
3. planner-service.MainVerticle   │
   - parses JSON body             │
   - validates userId, activityId
   - starts MySQL TRANSACTION ────┘
```

```
     INSERT INTO plans (...)
     INSERT INTO outbox_events (...)
   - COMMIT
                                  |
4. HTTP 201 returned to browser   |
   (Kafka not yet involved here)  |
                                  |
5. OutboxDispatcher (background, every 5 s)
   - SELECT unpublished outbox rows
   - KafkaProducer.send(plans.events.v1, userId, envelope)
   - UPDATE outbox_events SET published_at = NOW()
                                  |
6. PlanCreatedConsumer (notification-service)
   - receives Kafka record
   - INSERT IGNORE INTO consumed_events  ─── idempotency guard
   - INSERT INTO notifications            ─── side effect
   - consumer.commit()                   ─── advance Kafka offset
                                  |
7. React polls /api/notifications/user/:id every 10 s
   - sees the new notification
   - shows toast + updates list
```

---

3. How Kafka Events Flow End-to-End

```
AUTH-SERVICE  (best-effort direct publish)
──────────────────────────────────────────
Register endpoint succeeds
    └─► EventPublisher.publish("users.events.v1", userId, envelope)
            └─► KafkaProducer.send(...)  ← up to 3 retries with backoff
                    └─► topic: users.events.v1

USER-SERVICE  (consumer)
────────────────────────────────────────
UserRegisteredConsumer.handleRecord(record)
    ├─► parse + validate envelope
    ├─► INSERT IGNORE INTO consumed_events (eventId, "user-service")
    │       rowCount == 0?  ──► duplicate, skip
    │       rowCount == 1?  ──► proceed
    └─► INSERT INTO user_profiles (id, user_id, email, name)
            ON DUPLICATE KEY UPDATE ...
        consumer.commit()

PLANNER-SERVICE  (outbox pattern)
────────────────────────────────────────
```

```
createPlan HTTP handler
    └─► mysqlClient.withTransaction(conn → {
            INSERT INTO plans (...)
            INSERT INTO outbox_events (id, "Plan", planId, "PlanCreated",
envelope_json)
        })  ← single atomic transaction

OutboxDispatcher (periodic verticle, every 5 s)
    └─► SELECT id, aggregate_id, type, payload
          FROM outbox_events
          WHERE published_at IS NULL
          LIMIT 50
        for each row:
            EventPublisher.publish("plans.events.v1", planId, envelope)
                └─► on success: UPDATE outbox_events SET published_at = NOW()
                    on failure: leave NULL, retry next poll

NOTIFICATION-SERVICE  (consumer)
────────────────────────────────────────────
PlanCreatedConsumer.handleRecord(record)
    ├─► parse + validate envelope
    ├─► filter: only type == "PlanCreated"
    ├─► mysqlClient.withTransaction(conn → {
    │       INSERT IGNORE INTO consumed_events (eventId, "notification-
service")
    │       if rowCount == 1:
    │           INSERT INTO notifications (id, user_id, title, message,
"PLAN", false)
    │   })
    └─► consumer.commit()
```

---

4. Database Schema

File: `mysql/init.sql`

This file runs automatically when the MySQL Docker container starts for the first time (via `/docker-entrypoint-initdb.d/`). It creates all tables and inserts seed data.

Tables

`users`
Owned by **auth-service**. Every registered user has one row here.
```
id          VARCHAR(36) PK       — UUID generated by auth-service
```

```
email       VARCHAR(255) UNIQUE  — login credential
password    VARCHAR(255)         — plain-text (dev only!)
name        VARCHAR(255)
role        VARCHAR(50)          — USER | ADMIN
created_at  TIMESTAMP
updated_at  TIMESTAMP
```

`user_profiles`
Owned by **user-service** (created by the Kafka consumer, not HTTP).
```
id          VARCHAR(36) PK
user_id     VARCHAR(36) UNIQUE   — FK → users.id (CASCADE DELETE)
                                    UNIQUE ensures one profile per user
email       VARCHAR(255)
name        VARCHAR(255)
phone       VARCHAR(50)
created_at  TIMESTAMP
updated_at  TIMESTAMP
```
> The `UNIQUE KEY uq_user_profiles_user_id` was added specifically to support the
> idempotent `ON DUPLICATE KEY UPDATE` upsert in `UserRegisteredConsumer`.

`activities`
Owned by **activity-service**.
```
id               VARCHAR(36) PK
name             VARCHAR(255)
description      TEXT
category         VARCHAR(100)
duration         INT (minutes, default 60)
location         VARCHAR(255)
max_participants INT (default 10)
created_at       TIMESTAMP
updated_at       TIMESTAMP
```

`plans`
Owned by **planner-service**.
```
id             VARCHAR(36) PK
user_id        VARCHAR(36)         — which user made this plan
activity_id    VARCHAR(36)         — FK → activities.id (CASCADE DELETE)
activity_name  VARCHAR(255)        — denormalised snapshot
scheduled_date DATE
scheduled_time TIME
status         VARCHAR(50)         — SCHEDULED | CANCELLED | COMPLETED
```

```
notes          TEXT
created_at     TIMESTAMP
updated_at     TIMESTAMP
```

`notifications`
Owned by **notification-service** (rows created by `PlanCreatedConsumer`).
```
id         VARCHAR(36) PK
user_id    VARCHAR(36)         — FK → users.id (CASCADE DELETE)
title      VARCHAR(255)
message    TEXT
type       VARCHAR(50)         — INFO | PLAN | ALERT
is_read    BOOLEAN (default FALSE)
created_at TIMESTAMP
```

`outbox_events` ← NEW (Kafka)
Used exclusively by **planner-service** for the transactional outbox pattern.
```
id             VARCHAR(36) PK     — UUID
aggregate_type VARCHAR(100)       — "Plan"
aggregate_id   VARCHAR(36)        — planId
type           VARCHAR(100)       — PlanCreated | PlanUpdated | PlanCancelled
payload        JSON               — complete event envelope JSON
created_at     TIMESTAMP
published_at   TIMESTAMP NULL     — set by OutboxDispatcher on success
INDEX idx_outbox_unpublished (published_at, created_at)
```
> A row with `published_at IS NULL` is a pending event.
> The index makes the dispatcher poll fast even with millions of rows.

`consumed_events` ← NEW (Kafka)
Deduplication table shared by all Kafka consumers.
```
event_id       VARCHAR(36)   — from envelope.eventId
consumer_group VARCHAR(100)  — "user-service" | "notification-service"
processed_at   TIMESTAMP
PRIMARY KEY (event_id, consumer_group)
```
> Composite PK ensures each (eventId, consumerGroup) pair is processed once.
> `INSERT IGNORE` returns `rowCount=0` on replay → consumer skips processing.

---

5. Infrastructure Files

`docker-compose.yml`

Defines the entire local development environment as a single command (`docker compose up --build`).

Services declared:

Service	Image / Build	Port	Waits for
`mysql`	`mysql:8.0`	3306	—
`redpanda`	`redpandadata/redpanda:v23.1.11`	9092	—
`auth-service`	`./auth-service` (Dockerfile)	8081	mysql healthy, redpanda healthy
`user-service`	`./user-service`	8082	mysql healthy, redpanda healthy
`activity-service`	`./activity-service`	8083	mysql healthy, redpanda healthy
`planner-service`	`./planner-service`	8084	mysql healthy, redpanda healthy
`notification-service`	`./notification-service`	8085	mysql healthy, redpanda healthy
`api-gateway`	`./api-gateway`	8080	all 5 services

Key details:
- `mysql` has a healthcheck (`mysqladmin ping`) — services use `condition: service_healthy` so they don't start before MySQL is ready.
- `redpanda` has a healthcheck (`rpk cluster info`) — same pattern.
- All services share the `activity-network` bridge network so they can reach each other by service name (e.g., `mysql`, `redpanda`).
- `mysql_data` volume persists database data between `docker compose down` / `up` cycles.
- Each Java service receives `KAFKA_BOOTSTRAP_SERVERS=redpanda:9092` so Kafka clients know where to connect.

`mysql/init.sql`
SQL script mounted at `/docker-entrypoint-initdb.d/init.sql` inside the MySQL container. MySQL runs it automatically on first startup (when the data directory is empty).

What it does:
1. `USE activity_planner;` — selects the database created by `MYSQL_DATABASE` env var.
2. Creates all 7 tables with `IF NOT EXISTS` (safe to re-run).
3. Inserts 2 seed users (`admin@activity.com`, `user@activity.com`).
4. Inserts 2 sample activities (Morning Yoga, Team Building Workshop).

`<service>/Dockerfile`
Each service has an identical Dockerfile pattern:
```dockerfile
FROM openjdk:11-jdk-slim
WORKDIR /app
```

```
COPY target/<service>-1.0-SNAPSHOT-fat.jar /app/<service>.jar
EXPOSE <port>
ENV JAVA_OPTS="-Xms256m -Xmx512m"
ENTRYPOINT ["sh", "-c", "java $JAVA_OPTS -jar /app/<service>.jar"]
```

What it does:
- Uses Java 11 slim base image.
- Copies the Maven fat JAR (produced by `maven-shade-plugin`) into the container.
- Sets JVM heap limits via `JAVA_OPTS`.
- Runs the JAR with `java -jar` (Vert.x `Launcher` is the main class, embedded in the JAR manifest).

`<service>/pom.xml`
Maven build descriptor for each service. Key points:

Properties:
- `vertx.version = 4.0.0` — Vert.x version used across all services.
- `main.verticle = io.vertx.starter.MainVerticle` — the verticle class to launch.
- `maven.compiler.source/target = 11` — Java 11 compilation.

Dependencies (all services):

Dependency	Purpose
`vertx-core`	Core Vert.x: event loop, Futures, Vertx instance
`vertx-web`	HTTP router, `RoutingContext`, `BodyHandler`
`vertx-web-client`	HTTP client (used by api-gateway to forward requests)
`vertx-auth-jwt`	JWT creation and verification (auth-service)
`vertx-sql-client`	SQL connection pool abstractions
`vertx-mysql-client`	MySQL reactive driver
`vertx-kafka-client`	← NEW: Kafka producer/consumer over Vert.x event loop
`mysql-connector-j:8.0.33`	JDBC driver (loaded by vertx-mysql-client)

Test dependencies (all services):

Dependency	Purpose
`junit:4.13.2`	JUnit 4 test runner
`vertx-unit`	Async test support for Vert.x (`TestContext`, `Async`)
`rest-assured:4.3.3`	HTTP test assertions
`mockito-core:3.9.0`	Mocking framework

notification-service extra test deps:

Dependency	Purpose
`testcontainers:1.17.6`	Spins up real Docker containers in tests
`testcontainers/kafka:1.17.6`	Kafka (Confluent) container

```
| `testcontainers/mysql:1.17.6` | MySQL container |
| `kafka-clients:2.6.0` | Standard Kafka producer for test event injection |
| `slf4j-simple:1.7.36` | Logging backend required by Testcontainers |

**Build plugins:**
- `maven-shade-plugin` — merges all classes + dependencies into a single fat
JAR (`<artifactId>-1.0-SNAPSHOT-fat.jar`). Sets `Main-
Class=io.vertx.core.Launcher` and `Main-Verticle` in the JAR manifest so `java
-jar` knows what to start.
- `exec-maven-plugin` — lets you run the service with `mvn exec:java` during
development.

---

## 6. auth-service (Port 8081)

**Purpose:** User registration and login. Issues JWT tokens. Source of truth
for the `users` table. Emits `UserRegistered` Kafka events.

### `src/main/java/io/vertx/starter/MainVerticle.java`

The single Java class that forms the entire service. Extends
`AbstractVerticle` (Vert.x's base class).

**Fields:**
| Field | Type | Purpose |
|---|---|---|
| `jwtAuth` | `JWTAuth` | Vert.x JWT provider (HS256 algorithm) |
| `mysqlClient` | `MySQLPool` | Reactive MySQL connection pool (10
connections) |
| `eventPublisher` | `EventPublisher` | Kafka producer wrapper |
| `port` | `int` | 8081 |
| `dbHost/Port/Name/User/Password` | `String/int` | Read from environment
variables |

**`start(Promise<Void>)`** — called by Vert.x when the verticle is deployed:
1. Creates the MySQL connection pool.
2. Creates the JWT provider with a hardcoded HS256 secret (dev only).
3. Creates the `EventPublisher`.
4. Sets up an HTTP `Router` with:
   - A CORS handler on every route (allows all origins).
   - `GET /health` → returns `{"status":"UP"}`.
   - `POST /api/auth/register` → handled by `register()`.
   - `POST /api/auth/login` → handled by `login()`.
5. Starts the HTTP server on port 8081.

**`register(RoutingContext)`:**
1. Parses JSON body — requires `email`, `password`, `name`.
```

2. Checks `users` table for duplicate email.
3. `INSERT INTO users` with a new UUID.
4. Sends HTTP 201 response immediately.
5. Builds `UserRegistered` event envelope (does NOT include password).
6. Calls `eventPublisher.publish("users.events.v1", userId, envelope)` — fire-and-forget.

`login(RoutingContext)`:
1. Parses JSON body — requires `email`, `password`.
2. Queries `users` by email.
3. Compares plain-text password (dev-only limitation).
4. Generates JWT token with claims `{sub, email, name, role}`.
5. Returns `{token, user}`.

`stop(Promise<Void>)`:
Closes the Kafka producer (waits for flush), then closes the MySQL pool.

`src/main/java/io/vertx/starter/kafka/KafkaConfig.java`
Reads environment variables and produces a `Map<String, String>` for the Kafka producer.

`producerConfig()` returns:
```
bootstrap.servers  = KAFKA_BOOTSTRAP_SERVERS (default: localhost:9092)
security.protocol  = KAFKA_SECURITY_PROTOCOL (default: PLAINTEXT)
client.id          = KAFKA_CLIENT_ID          (default: auth-service)
key.serializer     = StringSerializer
value.serializer   = StringSerializer
acks               = 1  (leader acknowledgement)
linger.ms          = 5  (small batching window)
```
If `KAFKA_SECURITY_PROTOCOL` is `SASL_SSL`, it also adds `sasl.mechanism` and `sasl.jaas.config` from env vars.

`src/main/java/io/vertx/starter/kafka/EventEnvelope.java`
Utility class — builds the standard JSON event envelope.

`create(type, sourceService, data)` → returns:
```json
{
  "eventId":      "<new UUID>",
  "type":         "UserRegistered",
  "occurredAt":   "2026-03-24T10:00:00Z",
  "traceId":      "<new UUID>",
  "sourceService":"auth-service",
  "data":         { ... }
}
```

`withTraceId(type, sourceService, traceId, data)` — same but uses the supplied `traceId` (propagated from the HTTP `X-Trace-ID` header for end-to-end tracing).

`isValid(envelope)` — returns `false` if any required field is missing or if `data` is not a `JsonObject`. Used by consumers to reject malformed messages.

`src/main/java/io/vertx/starter/kafka/EventPublisher.java`
Wraps `KafkaProducer<String, String>` with retry logic.

`publish(topic, key, envelope)` → `Future<RecordMetadata>`
- Serialises the envelope to JSON string.
- Calls `KafkaProducerRecord.create(topic, key, value)`.
- Sends via `producer.send(record)`.
- On failure: waits 1s/2s/4s (exponential backoff) and retries up to 3 times.
- After 3 failures: fails the Future (caller decides how to handle).

`close()` → `Future<Void>` — flushes and closes the underlying Kafka producer.

---

7. user-service (Port 8082)

Purpose: CRUD operations on user profiles. Consumes `UserRegistered` events to create profiles (fixing the duplication bug where this service previously double-inserted into the `users` table).

`src/main/java/io/vertx/starter/MainVerticle.java`

`start(Promise<Void>)`:
1. Creates MySQL pool.
2. Deploys `UserRegisteredConsumer` as a **child verticle** — this starts the Kafka consumer in its own event loop context.
3. Sets up HTTP routes.

HTTP Routes:
Method	Path	Handler	What it does
GET	`/api/users`	`getAllUsers`	SELECT all from `user_profiles`
GET	`/api/users/:id`	`getUserById`	SELECT by profile `id`
POST	`/api/users`	`createUser`	INSERT into `user_profiles` only (requires `userId` of existing user)
PUT	`/api/users/:id`	`updateUser`	UPDATE `email/name/phone`
DELETE	`/api/users/:id`	`deleteUser`	DELETE from `user_profiles`

> **Bug fix:** The old `createUser` inserted into both `users` AND `user_profiles`. The new version only touches `user_profiles` and requires an existing `userId`. Profile creation in the normal registration flow is now exclusively handled by `UserRegisteredConsumer`.

`src/main/java/io/vertx/starter/kafka/KafkaConfig.java`
Identical structure to auth-service's version. Default `client.id` = `user-service`. Only exposes `consumerConfig(groupId)` (no producer needed).

`consumerConfig(groupId)` returns:
```
bootstrap.servers   = KAFKA_BOOTSTRAP_SERVERS
group.id            = groupId (passed in as "user-service")
auto.offset.reset   = earliest  (read from beginning on first start)
enable.auto.commit  = false     (manual commit after idempotent write)
max.poll.records    = 50
key/value.deserializer = StringDeserializer
```

`src/main/java/io/vertx/starter/kafka/EventEnvelope.java`
Same utility class as in auth-service. Includes `isValid()` for consumer-side validation.

`src/main/java/io/vertx/starter/kafka/UserRegisteredConsumer.java`
An `AbstractVerticle` deployed by `MainVerticle`.

`start(Promise<Void>)`:
1. Creates `KafkaConsumer<String, String>` using `KafkaConfig.consumerConfig("user-service")`.
2. Registers `exceptionHandler`, `partitionsRevokedHandler`, `partitionsAssignedHandler`.
3. Calls `consumer.handler(this::handleRecord)` — processes one record at a time (non-blocking).
4. Subscribes to `users.events.v1`.

`handleRecord(KafkaConsumerRecord)`:
```
1. Parse record.value() as JsonObject  → on error: log, commit, return
2. EventEnvelope.isValid(envelope)     → on invalid: log, commit, return
3. Check type == "UserRegistered"      → else: commit, return (skip other event types)
4. Extract: eventId, traceId, userId, email, name
5. Validate required fields            → on missing: log, commit, return
6. mysqlClient.withTransaction(conn → {
     INSERT IGNORE INTO consumed_events (eventId, "user-service")
     if rowCount == 0: already processed → skip
     if rowCount == 1:
```

```
       INSERT INTO user_profiles (...) ON DUPLICATE KEY UPDATE email=...,
name=...
   })
7. On tx success: consumer.commit()
   On tx failure: do NOT commit → record redelivered on next poll
```

Why not commit on failure?
If the DB write fails (e.g., MySQL is temporarily down), the Kafka offset is NOT advanced. On the next consumer poll, the same record is redelivered. The `consumed_events` guard ensures that even if the record is processed twice, the result is idempotent.

`stop(Promise<Void>)`: Closes the Kafka consumer gracefully.

---

8. activity-service (Port 8083)

Purpose: CRUD for activities. Emits `ActivityCreated`, `ActivityUpdated`, `ActivityDeleted` events (best-effort direct publish).

`src/main/java/io/vertx/starter/MainVerticle.java`

`start(Promise<Void>)`:
1. Creates MySQL pool.
2. Creates `EventPublisher`.
3. Sets up HTTP router + CORS.

HTTP Routes:
Method	Path	What it does
GET	`/api/activities`	SELECT all activities
GET	`/api/activities/:id`	SELECT by id
POST	`/api/activities`	INSERT + emit `ActivityCreated`
PUT	`/api/activities/:id`	UPDATE + emit `ActivityUpdated`
DELETE	`/api/activities/:id`	DELETE + emit `ActivityDeleted`

Pattern for each write:
1. Execute DB operation.
2. Send HTTP response.
3. Build event envelope using `EventEnvelope.withTraceId(...)` (extracts `X-Trace-ID` header).
4. Call `eventPublisher.publish(...)` — fire-and-forget (failure only logged).

`rowToJson(Row)` — private helper that converts a MySQL `Row` object into a `JsonObject` using the correct column names.

Kafka classes
Same `KafkaConfig.java`, `EventEnvelope.java`, `EventPublisher.java` as auth-service. Default `client.id` = `activity-service`.

---

9. planner-service (Port 8084)

Purpose: Manages activity plans. Uses the **transactional outbox pattern** to guarantee Kafka delivery even if the broker is temporarily unavailable.

`src/main/java/io/vertx/starter/MainVerticle.java`

`start(Promise<Void>)`:
1. Creates MySQL pool.
2. Creates `EventPublisher`.
3. Deploys `OutboxDispatcher` as a child verticle (passing the pool and publisher).
4. Sets up HTTP router.

HTTP Routes:
Method	Path	What it does
GET	`/api/plans`	SELECT all plans
GET	`/api/plans/user/:userId`	SELECT plans filtered by userId
GET	`/api/plans/:id`	SELECT plan by id
POST	`/api/plans`	Outbox transaction: INSERT plan + INSERT outbox row
PUT	`/api/plans/:id`	Outbox transaction: UPDATE plan + INSERT outbox row
DELETE	`/api/plans/:id`	Outbox transaction: DELETE plan + INSERT outbox row

`createPlan(RoutingContext)` — detailed flow:
```java
// Build event envelope BEFORE the transaction
JsonObject envelope = EventEnvelope.withTraceId("PlanCreated", "planner-service", traceId, eventData);

// Write plan + outbox in a single atomic transaction
mysqlClient.withTransaction(conn ->
  conn.preparedQuery("INSERT INTO plans ...").execute(...)
    .flatMap(r ->
      conn.preparedQuery("INSERT INTO outbox_events (id, aggregate_type, aggregate_id, type, payload) VALUES (?, ?, ?, ?, ?)")
        .execute(Tuple.of(outboxId, "Plan", planId, "PlanCreated", envelope.encode()))
    )
)
```

```
.onSuccess(r -> ctx.response().setStatusCode(201).end(...))
.onFailure(err -> ctx.response().setStatusCode(500).end(...));
```

Why is the envelope built before the transaction?
Because the `eventId` (UUID) must be the same in both the outbox row and the published Kafka message. Building it outside the transaction ensures consistency.

`src/main/java/io/vertx/starter/kafka/OutboxDispatcher.java`
An `AbstractVerticle` that runs a periodic polling loop.

`start(Promise<Void>)`:
Registers a Vert.x periodic timer: `vertx.setPeriodic(5000, id -> dispatch())`.

`dispatch()` — called every 5 seconds:
```
1. SELECT id, aggregate_id, type, payload
   FROM outbox_events
   WHERE published_at IS NULL
   ORDER BY created_at
   LIMIT 50

2. For each row:
   a. Parse payload JSON
   b. eventPublisher.publish("plans.events.v1", aggregateId, envelope)
      → on success: UPDATE outbox_events SET published_at = NOW() WHERE id = ?
      → on failure: log error, leave published_at NULL (will retry next poll)
   c. If payload is unparseable (malformed JSON):
      → mark as published (dead-letter) to avoid infinite retries
      → in production, write to a dead-letter table instead

3. On DB query failure: log error, skip this poll cycle
```

Ordering guarantee: The `ORDER BY created_at` ensures events for the same plan are published in insertion order.

`stop(Promise<Void>)`: Closes the `EventPublisher` (flushes Kafka producer buffer).

Kafka classes
`KafkaConfig.java`, `EventEnvelope.java`, `EventPublisher.java` — same as auth-service. Default `client.id` = `planner-service`.

---

10. notification-service (Port 8085)

Purpose: Stores and serves notifications. Consumes `PlanCreated` events to automatically create notifications when a plan is made.

`src/main/java/io/vertx/starter/MainVerticle.java`

`start(Promise<Void>)`:
1. Creates MySQL pool.
2. Deploys `PlanCreatedConsumer` as a child verticle.
3. Sets up HTTP router.

HTTP Routes:
Method	Path	What it does
GET	`/api/notifications`	SELECT all notifications (newest first)
GET	`/api/notifications/user/:userId`	SELECT by userId (newest first)
POST	`/api/notifications`	Manual notification creation
PUT	`/api/notifications/:id/read`	Mark a notification as read
DELETE	`/api/notifications/:id`	Delete a notification

`src/main/java/io/vertx/starter/kafka/PlanCreatedConsumer.java`
An `AbstractVerticle` — mirror image of `UserRegisteredConsumer` but for `plans.events.v1`.

`handleRecord(KafkaConsumerRecord)`:
```
1. Parse JSON → validate → check type == "PlanCreated"
2. Extract: eventId, traceId, userId, activityName, scheduledDate
3. mysqlClient.withTransaction(conn → {
     INSERT IGNORE INTO consumed_events (eventId, "notification-service")
     if rowCount == 0: already processed → skip
     if rowCount == 1:
       INSERT INTO notifications (id, user_id, title, message, "PLAN", false)
       title   = "Plan Created"
       message = 'Your plan for "Morning Yoga" on 2026-04-01 has been
created.'
   })
4. On success: consumer.commit()
   On failure: do NOT commit → redelivered
```

Kafka classes
`KafkaConfig.java`, `EventEnvelope.java` (includes `withTraceId`) — same pattern. Default `client.id` = `notification-service`. Only `consumerConfig` needed (no producer).

---

11. api-gateway (Port 8080)

Purpose: The single public entry point. Every request from the browser goes through here. It proxies requests to the correct downstream service using Vert.x `WebClient`.

`src/main/java/io/vertx/starter/MainVerticle.java`

Fields:
- Service host addresses (hardcoded to Docker service names):
  ```
  authServiceHost     = localhost:8081
  userServiceHost     = localhost:8082
  activityServiceHost = localhost:8083
  plannerServiceHost  = localhost:8084
  notifServiceHost    = localhost:8085
  ```
- `WebClient client` — Vert.x HTTP client for forwarding requests.

`start(Promise<Void>)`:
1. Creates a `WebClient`.
2. Sets up a Router with CORS headers.
3. Serves the React frontend's static files.
4. Registers route matchers that proxy to backend services:
   - `/api/auth/*` → auth-service:8081
   - `/api/users/*` → user-service:8082
   - `/api/activities/*` → activity-service:8083
   - `/api/plans/*` → planner-service:8084
   - `/api/notifications/*` → notification-service:8085

Proxy mechanism:
- Copies the HTTP method, path, body, and headers from the incoming request.
- Sends it to the target service using `WebClient`.
- Copies the response status, headers, and body back to the original caller.
- On connection failure: returns HTTP 503 Service Unavailable.

No Kafka involvement — the gateway is purely a transparent HTTP proxy.

---

12. Frontend (React SPA)

Directory: `frontend/`

`src/App.js`

The entire React application lives in this single file. It exports a default `App` component and defines all pages inline.

`AuthContext` — a React Context object that makes auth state available to all components without prop-drilling.
- `user` — the currently logged-in user object `{sub, email, name, role}` (decoded from JWT).
- `token` — the raw JWT string (stored in `localStorage`).
- `login(email, password)` — calls `POST /api/auth/login`, stores token.
- `register(email, password, name)` — calls `POST /api/auth/register`.
- `logout()` — clears `localStorage` and resets state.
- `showToast(message, type)` — shows a toast notification.

`Toast` component:
- A fixed-position notification box (bottom-right).
- Auto-dismisses after 3.5 seconds.
- Types: `success` (green), `error` (red), `info` (blue).

`useToast()` hook:
- Manages toast state (`{message, type}`).
- Returns `{ toast, show, hide }`.

`App` component (root):
- Wraps everything in `AuthContext.Provider` and `BrowserRouter`.
- Renders `<nav>` with conditional links (login/logout depending on auth state).
- Defines routes: `/`, `/login`, `/register`, `/dashboard`, `/activities`, `/plans`, `/notifications`.
- Protected routes redirect to `/login` if not authenticated.
- Renders `<Toast>` when toast state is set.

`Login` component:
- Form with email + password fields.
- Calls `AuthContext.login()`.
- Shows error message on failure.

`Register` component:
- Form with name + email + password.
- Calls `AuthContext.register()`.
- Shows success message on completion (does not auto-login).

`Dashboard` component:
- Shows count of available activities and the user's plans.
- Fetches both in parallel with `Promise.all`.

`PlanModal` component: ← NEW
- A modal dialog that opens when "Plan This Activity" is clicked.
- Fields: date, time, notes.

- On submit: calls `POST /api/plans` with `{userId, activityId, activityName, scheduledDate, scheduledTime, notes}`.
- On success: shows toast "Plan created for X!", closes modal.
- On error: shows toast with the server error message.

`Activities` component:
- Fetches and displays all activities in a card grid.
- Each card has a **"Plan This Activity" button** ← NOW WIRED (was broken before).
- Clicking the button opens `PlanModal` with that activity pre-filled.

`Plans` component:
- Fetches plans for the current user (`GET /api/plans/user/:userId`).
- Displays in a table: activity name, date, time, status, notes.

`Notifications` component: ← ENHANCED
- Fetches notifications for the current user.
- **Polls every 10 seconds** using `setInterval` in a `useEffect`.
- On each poll, compares new results to previous state.
- If new unread notifications appeared: shows a toast badge.
- "Mark as Read" button calls `PUT /api/notifications/:id/read`.
- The interval is cleared on component unmount (no memory leaks).

`frontend/Dockerfile`
Multi-stage build:
1. **Stage 1 (build):** `node:14-alpine` — runs `npm install && npm run build` to produce `/app/build/`.
2. **Stage 2 (serve):** `nginx:alpine` — copies the `/app/build/` output and `nginx.conf`.

`frontend/nginx.conf`
Nginx configuration:
- Serves static files from `/usr/share/nginx/html`.
- `try_files $uri $uri/ /index.html` — SPA routing (all unknown paths serve `index.html`, React Router handles the rest).
- `location /api { proxy_pass http://api-gateway:8080; }` — API calls from the browser are proxied to the gateway.

`frontend/package.json`
Node.js project metadata.
- `react`, `react-dom`: 17.0.2
- `react-router-dom`: 5.3.0 (uses `<Routes>` and `<Route>` from v6-style API via the v5 compat layer)
- `axios`: 0.21.4 — HTTP client for API calls

---

13. Kafka Infrastructure Classes

These classes exist in a `kafka/` sub-package inside each relevant service. They are intentionally duplicated (each service is an independent Maven module with no shared library).

`KafkaConfig.java` (in auth, activity, planner, user, notification services)

Method	Present in	Returns
`producerConfig()`	auth, activity, planner	`Map<String,String>` for `KafkaProducer.create()`
`consumerConfig(groupId)`	user, notification	`Map<String,String>` for `KafkaConsumer.create()`

Reads from environment: `KAFKA_BOOTSTRAP_SERVERS`, `KAFKA_SECURITY_PROTOCOL`, `KAFKA_CLIENT_ID`. SASL_SSL branch also reads `KAFKA_SASL_MECHANISM`, `KAFKA_SASL_USERNAME`, `KAFKA_SASL_PASSWORD`.

`EventEnvelope.java` (in all 5 services)

Method	Purpose
`create(type, source, data)`	Builds envelope with auto-generated traceId
`withTraceId(type, source, traceId, data)`	Builds envelope propagating an existing traceId
`isValid(envelope)`	Validates all required fields exist and `data` is a JsonObject

`EventPublisher.java` (in auth, activity, planner services)

Wraps `io.vertx.kafka.client.producer.KafkaProducer<String, String>`.

Method	Purpose
`publish(topic, key, envelope)`	Returns `Future<RecordMetadata>`, retries up to 3× with backoff
`close()`	Returns `Future<Void>`, flushes and closes the producer

`OutboxDispatcher.java` (planner-service only)

Extends `AbstractVerticle`. Injected with `MySQLPool` and `EventPublisher` in constructor.

Method	Purpose
`start(Promise<Void>)`	Registers a `vertx.setPeriodic(5000, ...)` timer
`dispatch()`	Polls outbox, publishes, marks published

| `markPublished(outboxId)` | `UPDATE outbox_events SET published_at = NOW()` |
| `stop(Promise<Void>)` | Closes `EventPublisher` |

`UserRegisteredConsumer.java` (user-service only)

Extends `AbstractVerticle`. Injected with `MySQLPool`.

Subscribes to `users.events.v1` with consumer group `user-service`.

`PlanCreatedConsumer.java` (notification-service only)

Extends `AbstractVerticle`. Injected with `MySQLPool`.

Subscribes to `plans.events.v1` with consumer group `notification-service`.

---

14. Kubernetes Manifests

Directory: `k8s/`

All manifests use `apiVersion: apps/v1` (Deployment) and `apiVersion: v1` (Service).

`k8s/kafka.yaml`
Deploys Redpanda (Kafka-compatible, no Zookeeper) as a `StatefulSet`.

Resources declared:
1. `StatefulSet/redpanda` (1 replica)
   - Uses `docker.redpanda.com/redpandadata/redpanda:v23.1.11`.
   - Readiness/Liveness probes run `rpk cluster info`.
   - `VolumeClaimTemplate` requests 5 Gi persistent storage.
2. `Service/kafka` (headless, `clusterIP: None`)
   - DNS name inside the cluster: `kafka.default.svc.cluster.local` (short: `kafka`).
   - Port 9092.
3. `Service/kafka-external` (NodePort 30092)
   - For `kubectl port-forward` or direct minikube access.

`k8s/auth-service-deployment.yaml`
Deployment: 1 replica of `activity-planner/auth-service:latest`.
- Environment: `JAVA_OPTS`, `DB_HOST=mysql`, `DB_USER/DB_PASSWORD` from `mysql-credentials` Secret, `KAFKA_BOOTSTRAP_SERVERS=kafka:9092`, `KAFKA_CLIENT_ID=auth-service`.

Service: ClusterIP on port 8081.

> Same pattern applies to `user-service-deployment.yaml`, `activity-service-deployment.yaml`, `planner-service-deployment.yaml`, `notification-service-deployment.yaml`.

`k8s/api-gateway-deployment.yaml`
Service type: `LoadBalancer` (port 80 → 8080) — exposes the gateway externally.

---

15. Documentation Files

`docs/kafka-events.md`
The canonical event schema reference.
- Topic table (name, key, partitions, retention).
- Full JSON samples for every event type (`UserRegistered`, `ActivityCreated/Updated/Deleted`, `PlanCreated/Updated/Cancelled`).
- Backward-compatibility rules.
- Outbox pattern diagram.
- Idempotency strategy.
- Scaling / partition count recommendations.
- Migration path to Avro.
- SASL_SSL configuration snippet.

`docs/run-local-with-kafka.md`
Step-by-step guide for local development.
- How to start the stack (`docker compose up --build`).
- Environment variables table.
- How to list and tail Kafka topics with `rpk`.
- How to trigger events via `curl`.
- How to inspect the outbox and consumed_events tables.
- How to produce a test event manually.

`docs/k8s-kafka-notes.md`
Kubernetes deployment guide.
- Dev setup with minikube/kind.
- Bootstrap address inside the cluster.
- Strimzi CRD examples for production (`Kafka`, `KafkaTopic`, `KafkaUser`).
- SASL_SSL K8s Secret configuration.
- ACLs and quotas.
- Dev → Production checklist.

`README.md`
Project introduction (pre-existing, not modified).

`TODO.md` / `TODO_MySQL_Migration.md`
Pre-existing task tracking files (not modified).

---

16. Test Files

`notification-service/src/test/java/io/vertx/starter/EventEnvelopeTest.java`

Type: Unit test (no containers, no network).
Runner: JUnit 4.
Class under test: `io.vertx.starter.kafka.EventEnvelope`.

Tests:

Test	What it verifies
`createPopulatesAllRequiredFields`	All 6 required envelope fields are present and correct
`eachCallProducesUniqueEventId`	Two calls produce different `eventId` UUIDs
`withTraceIdPreservesSuppliedTraceId`	Supplied `traceId` is stored verbatim
`withTraceIdGeneratesTraceIdWhenNull`	`null` traceId → auto-generated UUID
`isValidAcceptsWellFormedEnvelope`	Valid envelope → `true`
`isValidRejectsNull`	`null` → `false`
`isValidRejectsMissingType`	Envelope without `type` → `false`
`isValidRejectsMissingEventId`	Envelope without `eventId` → `false`
`isValidRejectsNonObjectData`	`data` is a string, not an object → `false`
`isValidRejectsMissingData`	No `data` field → `false`
`occurredAtIsIso8601Utc`	`occurredAt` ends with `Z` and contains `T`

`notification-service/src/test/java/io/vertx/starter/NotificationKafkaIntegrationTest.java`

Type: Integration test using real MySQL + Kafka containers.
Runner: `@RunWith(VertxUnitRunner.class)` — Vert.x async test runner.
Containers (ClassRule — started once, shared across tests):
- `KafkaContainer` (`confluentinc/cp-kafka:6.2.1`) — real Kafka broker.
- `MySQLContainer` (`mysql:8.0`) — real MySQL database.

`@BeforeClass setup(TestContext)`:
1. Creates a `Vertx` instance.
2. Creates a `MySQLPool` pointing to the test container (using mapped port from Testcontainers).
3. Creates all required tables via async SQL queries (`users`, `notifications`, `consumed_events`).
4. Inserts a test user row (needed for the `notifications.user_id` FK).

5. Deploys `TestablePlanCreatedConsumer` (subclass that injects test Kafka bootstrap).
6. Creates a standard Java `KafkaProducer` for sending test events.

Test 1: `testPlanCreatedEventCreatesNotification`
1. Build a valid `PlanCreated` event envelope with a new UUID `eventId`.
2. Produce it to `plans.events.v1` using the test Kafka producer.
3. Poll `consumed_events` every 1 second (up to 10 s) until the row appears.
4. Assert that a `notifications` row also exists for the test user.

Test 2: `testIdempotentReplay`
1. Build a valid envelope.
2. Send it **twice** to Kafka.
3. Wait 5 seconds for processing.
4. Assert `consumed_events` has **exactly 1 row** for that `eventId` (not 2).

`TestablePlanCreatedConsumer` inner class:
Extends `PlanCreatedConsumer`. Overrides `start()` to set `KAFKA_BOOTSTRAP_SERVERS` as a system property before calling `super.start()`. This makes `KafkaConfig.consumerConfig()` pick up the Testcontainers Kafka address instead of the `localhost:9092` default.

`pollForNotification(...)` helper:
Recursive polling using `vertx.setTimer` — avoids `Thread.sleep` (which would block the Vert.x event loop).

---

17. Process Reference: What Happens When…

… a new user registers

```
Browser: POST /api/auth/register {email, password, name}
  → api-gateway proxies to auth-service:8081
  → auth-service checks for duplicate email in users table
  → auth-service INSERTs into users table
  → auth-service responds HTTP 201 to browser
  → auth-service publishes UserRegistered to users.events.v1 (async, best-effort)
  → user-service.UserRegisteredConsumer receives the event
  → user-service checks consumed_events for duplicate (INSERT IGNORE)
  → user-service INSERTs into user_profiles
  → user-service commits Kafka offset
```

… a plan is created (full Kafka path)

```
Browser: POST /api/plans {userId, activityId, ...}
  → api-gateway proxies to planner-service:8084
  → planner-service opens MySQL transaction
  → planner-service INSERTs into plans table
  → planner-service INSERTs into outbox_events (payload = PlanCreated
envelope)
  → planner-service COMMITS transaction
  → planner-service responds HTTP 201 to browser
  ↓ (up to 5 seconds later)
  → OutboxDispatcher polls outbox_events WHERE published_at IS NULL
  → OutboxDispatcher calls KafkaProducer.send(plans.events.v1, userId,
envelope)
  → Kafka broker acknowledges
  → OutboxDispatcher UPDATE outbox_events SET published_at = NOW()
  ↓ (milliseconds later, Kafka delivers to consumer)
  → notification-service.PlanCreatedConsumer receives the event
  → PlanCreatedConsumer runs idempotency check (INSERT IGNORE consumed_events)
  → PlanCreatedConsumer INSERTs into notifications
  → PlanCreatedConsumer commits Kafka offset
  ↓ (within 10 seconds)
  → React Notifications page polls GET /api/notifications/user/:id
  → New notification appears in the list
  → Toast shows "1 new notification"
```

… the same Kafka event is delivered twice (crash-recovery scenario)

```
Consumer receives event (first delivery)
  → transaction starts
  → INSERT IGNORE consumed_events (eventId, group) → rowCount = 1 (new)
  → INSERT notifications ...
  → COMMIT
  → ** consumer crashes before consumer.commit() **

Kafka redelivers the same offset:
  → Consumer receives same event again
  → transaction starts
  → INSERT IGNORE consumed_events (eventId, group) → rowCount = 0 (duplicate!)
  → skip business logic
  → COMMIT (no-op)
  → consumer.commit()
  → Result: exactly one notification row, correct
```

… Docker Compose starts up

```
1. MySQL container starts, runs healthcheck (mysqladmin ping)
   MySQL runs init.sql → creates tables + seed data
   MySQL reports healthy

2. Redpanda starts, runs healthcheck (rpk cluster info)
   Redpanda reports healthy

3. All 5 Java services start in parallel (once both healthchecks pass)
   Each service:
     a. JVM starts, Launcher loads MainVerticle class
     b. MainVerticle.start() called by Vert.x
     c. MySQLPool created (connects to mysql:3306)
     d. KafkaProducer/Consumer created (connects to redpanda:9092)
     e. HTTP server binds to its port
     f. Service logs "X Service started on port Y"

4. api-gateway starts (once all services are up)
   Binds to port 8080

5. Stack is ready — browser can connect to http://localhost:8080
```

---

18. Environment Variables Reference

Variable	Default	Used by	Purpose
`DB_HOST`	`localhost`	All services	MySQL hostname
`DB_PORT`	`3306`	All services	MySQL port
`DB_NAME`	`activity_planner`	All services	MySQL database name
`DB_USER`	`activity_user`	All services	MySQL username
`DB_PASSWORD`	`activity_password`	All services	MySQL password
`KAFKA_BOOTSTRAP_SERVERS`	`localhost:9092`	auth, activity, planner, user, notification	Kafka broker address
`KAFKA_SECURITY_PROTOCOL`	`PLAINTEXT`	Same	`PLAINTEXT` or `SASL_SSL`
`KAFKA_CLIENT_ID`	`<service-name>`	Same	Kafka client identifier
`KAFKA_SASL_MECHANISM`	`SCRAM-SHA-512`	Same (SASL_SSL only)	SASL mechanism
`KAFKA_SASL_USERNAME`	(empty)	Same (SASL_SSL only)	SASL username
`KAFKA_SASL_PASSWORD`	(empty)	Same (SASL_SSL only)	SASL password
`JAVA_OPTS`	`-Xms256m -Xmx512m`	All Java containers	JVM heap configuration
`MYSQL_ROOT_PASSWORD`	`Gyanendra9931@`	MySQL container	Root DB password
`MYSQL_DATABASE`	`activity_planner`	MySQL container	Database to create

| `MYSQL_USER` | `activity_user` | MySQL container | App DB user |
| `MYSQL_PASSWORD` | `Gyanendra9931@` | MySQL container | App DB password |

---

19. How to Build and Run

Local development (Docker Compose)

```bash
# 1. Build all fat JARs
cd Activity_Planner1

# Build each service
cd auth-service && mvn clean package -DskipTests && cd ..
cd user-service && mvn clean package -DskipTests && cd ..
cd activity-service && mvn clean package -DskipTests && cd ..
cd planner-service && mvn clean package -DskipTests && cd ..
cd notification-service && mvn clean package -DskipTests && cd ..
cd api-gateway && mvn clean package -DskipTests && cd ..

# 2. Start the full stack
docker compose up --build

# 3. Open browser
#    http://localhost:8080
#    Login: admin@activity.com / admin123
```

Run a single service locally (without Docker)

```bash
export DB_HOST=localhost
export DB_PASSWORD=Gyanendra9931@
export KAFKA_BOOTSTRAP_SERVERS=localhost:9092
cd auth-service
mvn exec:java
```

Run the integration tests

```bash
# Requires Docker running (Testcontainers pulls images automatically)
cd notification-service
mvn test
```

Run the unit tests only

```bash
cd notification-service
mvn test -Dtest=EventEnvelopeTest
```

Kubernetes (minikube)

```bash
# Start cluster
minikube start --memory=4096 --cpus=2

# Build images into minikube's Docker daemon
eval $(minikube docker-env)
docker build -t activity-planner/auth-service:latest ./auth-service
# ... repeat for each service

# Deploy
kubectl apply -f k8s/

# Access gateway
minikube service api-gateway --url
```

Tail Kafka topics (Docker Compose)

```bash
# Live tail of plan events
docker exec activity_planner1-redpanda-1 \
  rpk topic consume plans.events.v1 --brokers localhost:9092 --offset start

# Check outbox status
docker exec activity_planner1-mysql-1 \
  mysql -u activity_user -pGyanendra9931@ activity_planner \
  -e "SELECT id, type, published_at FROM outbox_events ORDER BY created_at
DESC LIMIT 10;"
```